

**ОТЧЁТ**

по лабораторной работе № 6

**ЧТО ПРОИСХОДИТ С BACKEND-СЕРВИСОМ ПОСЛЕ  
ШАРДИРОВАНИЯ**

по модулю «Масштабирование баз данных»

Исполнитель:

обучающийся группы K0709-23/3

\_\_\_\_\_  
подпись, дата

У. И. Усачева

Руководитель:

преподаватель

\_\_\_\_\_  
подпись, дата

\_\_\_\_\_

## СОДЕРЖАНИЕ

|                                                   |    |
|---------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                     | 3  |
| 1 Архитектура после шардирования.....             | 6  |
| 2 Запрос к одному шарду .....                     | 7  |
| 3 Агрегирующий запрос .....                       | 8  |
| 4 JOIN между шардами.....                         | 11 |
| 5 ORDER BY и LIMIT .....                          | 14 |
| 6 Отказ одного шарда .....                        | 16 |
| 7 Горячий шард .....                              | 19 |
| 8 Итоговый анализ shard key.....                  | 21 |
| 8.1 Плюсы .....                                   | 21 |
| 8.2 Минусы .....                                  | 21 |
| 8.3 Проблемные операции.....                      | 21 |
| 8.4 Вывод .....                                   | 22 |
| 9 Ответы на вопросы для самопроверки.....         | 24 |
| ЗАКЛЮЧЕНИЕ .....                                  | 28 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....            | 30 |
| ПРИЛОЖЕНИЕ А Сервис распределённых запросов ..... | 31 |
| ПРИЛОЖЕНИЕ Б Объединение результатов шардов ..... | 36 |
| ПРИЛОЖЕНИЕ В Запросы к отдельному шарду.....      | 39 |

## ВВЕДЕНИЕ

В лабораторной работе № 5 задачи сервиса TaskManager были распределены между тремя экземплярами PostgreSQL по ключу `user_id`. Шардирование решает проблему объёма данных, но меняет работу обычных запросов: то, что раньше выполнял один SQL-запрос к одной базе, теперь может требовать обращения к нескольким серверам.

Цель работы — понять, как шардирование изменяет работу SQL-запросов собственного сервиса и какие новые проблемы появляются при распределении данных между несколькими PostgreSQL.

Для достижения цели решались следующие задачи:

- найти в сервисе запрос, который выполняется на одном шарде, и объяснить почему;
- выполнить агрегирующий запрос по всем шардам и проверить правильность объединения результатов;
- разобрать JOIN сервиса, связанные данные которого после шардирования лежат в разных базах;
- выполнить `ORDER BY ... LIMIT` по всем шардам и оценить стоимость глубоких страниц;
- остановить один шард на работающем сервисе и проверить, что продолжает работать;
- воспроизвести горячий шард и сравнить распределение данных и нагрузки;
- оценить, подходит ли выбранный `shard key` для сервиса.

При выполнении использовались документация PostgreSQL и Npgsql [1–4], документация Citus [5], книга М. Клеппмана [6], методические указания [7] и инструмент записи терминала VHS [8].

Таблица 1 – Среда выполнения экспериментов

| Параметр | Значение                                                           |
|----------|--------------------------------------------------------------------|
| СУБД     | PostgreSQL 16.14 (postgres:16-alpine) в Docker Desktop, Windows 11 |

| Параметр      | Значение                                                                                      |
|---------------|-----------------------------------------------------------------------------------------------|
| Шарды         | postgres-shard-0...2 (порты 5440–5442), postgres-shard-3 (5443) для опыта с четырьмя шардами  |
| Router        | consistent hashing, 500 виртуальных узлов, shard key user_id (ЛР № 5)                         |
| Основная база | postgres (5432): пользователи, проекты, теги, связи задач с тегами, топология шардов          |
| Сервис        | TaskManager: .NET 8, EF Core 8, Dapper, Npgsql                                                |
| Данные        | 1 000 000 задач 10 000 пользователей: Shard 0 — 339 959, Shard 1 — 319 756, Shard 2 — 340 285 |

Все эксперименты автоматизированы:

Таблица 2 – Скрипты и результаты

| Раздел                 | Запуск                                  | Результаты                              |
|------------------------|-----------------------------------------|-----------------------------------------|
| 2 — single-shard query | bash scripts/lab-06/01-single-shard.sh  | docs/lab-06/results/01-single-shard.txt |
| 3 — агрегация          | bash scripts/lab-06/02-aggregation.sh   | 02-aggregation.txt                      |
| 4 — JOIN               | bash scripts/lab-06/03-join.sh          | 03-join.txt                             |
| 5 — ORDER BY + LIMIT   | bash scripts/lab-06/04-order-limit.sh   | 04-order-limit.txt                      |
| 6 — отказ шарда        | bash scripts/lab-06/05-shard-failure.sh | 05-shard-failure.txt                    |
| 7 — hot shard          | bash scripts/lab-06/06-hot-shard.sh     | 06-hot-shard.txt                        |
| Всё подряд             | bash scripts/lab-06/run-all.sh          | docs/lab-06/results/                    |
| Запись терминала       | vhs scripts/lab-06/demo.tape            | docs/lab-06/demo.gif                    |

**Как измерялось.** Распределённые запросы выполняет сам сервис (DistributedTaskQueryService, endpoint'ы /api/shards/queries/...): он отправляет запрос на нужные шарды, замеряет время каждого и объединяет ответы. Правильность объединения проверяется тем же запросом к основной базе, где все задачи по-прежнему лежат одной таблицей. Нагрузку для опыта с hot shard создаёт сам сервис, потому что внешний API ограничен 100 запросами в минуту. Независимое подтверждение нагрузки — счётчик транзакций pg\_stat\_database на каждом шарде.

**Запись терминала.** GIF снят инструментом VHS на работающем сервисе: COUNT и ORDER BY + LIMIT по всем шардам, остановка Shard 2, ответы сервиса и восстановление.

```

> powershell -ExecutionPolicy Bypass -File scripts/lab-06/demo-distributed.ps1 -Pause 3

=== COUNT по всем шардам: каждый шард считает свою часть, сервис складывает
shard-0: 339959 задач
shard-1: 319756 задач
shard-2: 340285 задач
Итого: 1000000 задач за 66,84 мс

=== ORDER BY created_at DESC LIMIT 5: top-5 с каждого шарда, слияние в сервисе
2026-09-13 22:50:56 Shard 2 Task #839460
2026-09-13 22:50:34 Shard 2 Task #91296
2026-09-13 22:50:03 Shard 1 Task #497509
2026-09-13 22:49:05 Shard 2 Task #606862
2026-09-13 22:48:24 Shard 1 Task #203638

=== Отказ: останавливаем Shard 2
docker compose stop postgres-shard-2: остановлен
Задачи gen_user_2 (Shard 0): HTTP 200
Задачи gen_user_4 (Shard 2): HTTP 503 SHARD_UNAVAILABLE
Статистика по всем шардам: HTTP 503 SHARD_UNAVAILABLE
Статистика, allowPartial=true: partial=True, total=659715
Проекты (основная база): HTTP 200

=== Восстановление: запускаем Shard 2
docker compose start postgres-shard-2: запущен
Статистика по всем шардам: HTTP 200, total=1000000
>

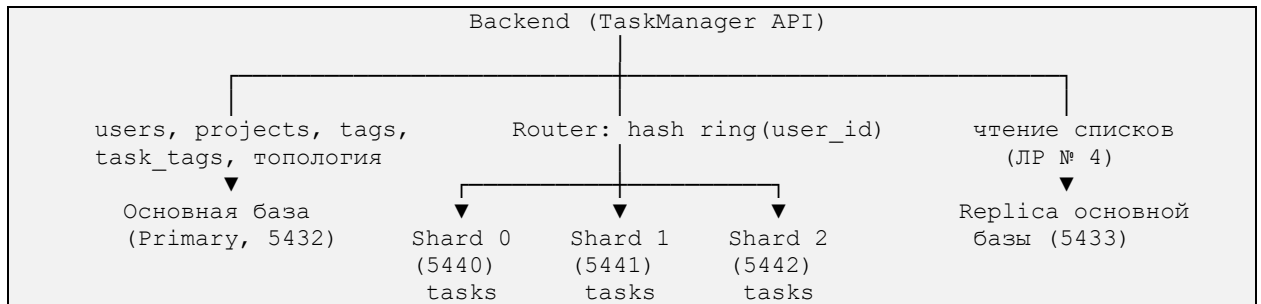
```

Рисунок 1 – Агрегация и ORDER BY + LIMIT по трём шардам, отказ Shard 2: 503 для его данных, неполная статистика по запросу и работа остальной части сервиса

## 1 Архитектура после шардирования

После ЛР № 4 и № 5 сервис работает уже не с одной базой, а с пятью экземплярами PostgreSQL. Router находится внутри сервиса: по `user_id` он выбирает шард для задач, а всё остальное идёт в основную базу или её реплику.

Листинг 1 – Архитектура TaskManager после шардирования



Из этого сразу видно, какие запросы будут простыми: те, что знают `user_id` и касаются только задач, идут на один шард. Всё остальное — запросы по всем задачам и запросы, которым вместе с задачами нужны проекты, пользователи или теги, — становится распределённым.

## 2 Запрос к одному шарду

**Запрос сервиса.** Список задач пользователя — GET /api/tasks от имени пользователя, метод `TaskRepository.GetByIdAsync`:

### Листинг 2 – Запрос списка задач пользователя

```
SELECT t.id, t.created_at, t.description, t.due_date, t.priority, t.project_id, t.status,
t.title, t.updated_at, t.user_id
FROM tasks AS t
WHERE t.user_id = $1
ORDER BY t.created_at DESC
LIMIT $2 OFFSET $3
```

После шардирования этот запрос выполняет GET /api/shards/users/{userId}/tasks. Router вычисляет шард пользователя `gen_user_100` — Shard 2 — и запрос уходит только туда.

Таблица 3 – Задачи пользователя `gen_user_100` на каждом шарде (SQL на шардах)

| Шард    | Задач пользователя |
|---------|--------------------|
| Shard 0 | 0                  |
| Shard 1 | 0                  |
| Shard 2 | 497                |

### Листинг 3 – План запроса на Shard 2

```
Limit (actual time=0.020..0.025 rows=20 loops=1)
  Buffers: shared hit=5
  -> Index Scan using idx_tasks_user_id_created_at on tasks (actual time=0.019..0.023 rows=20
      loops=1)
        Index Cond: (user_id = '462b8287-da8e-4d42-9496-8c0a9653c776'::uuid)
Execution Time: 0.071 ms
```

**Почему запросу не нужны другие шарды.** Условие `WHERE user_id = $1` содержит `shard key`. Router — детерминированная функция от этого ключа, поэтому по `user_id` можно заранее сказать, где лежат все задачи пользователя: при записи каждая его задача попала на тот же шард. На других шардах этих строк нет по построению, и SQL на них это подтверждает — 0 строк. Запрос выполняется как на обычной базе: один сервер, один индекс, 5 буферов и 0,07 мс на шарде, 10–50 мс через API.

### 3 Агрегирующий запрос

**Запрос сервиса.** Статистика задач по статусам — GET  
/api/tasks/stats, метод TaskRepository.GetStatusStatsAsync:

#### Листинг 4 – Агрегирующий запрос сервиса

```
SELECT status, count(*), count(*) FILTER (WHERE due_date < now() AND status IN (0, 1))  
FROM tasks  
GROUP BY status
```

После шардирования его выполняет GET  
/api/shards/queries/stats: запрос уходит на каждый шард, а сервис складывает ответы. Для среднего возраста задач шарды возвращают не средние, а сумму и количество, иначе объединение было бы неверным.

Таблица 4 – Ответы шардов и результат объединения

| Шард              | Задач            | Строк в ответе шарда   |
|-------------------|------------------|------------------------|
| Shard 0           | 339 959          | 4 (по одной на статус) |
| Shard 1           | 319 756          | 4                      |
| Shard 2           | 340 285          | 4                      |
| После объединения | <b>1 000 000</b> | 4                      |

Таблица 5 – Результат по статусам после объединения и проверка на основной базе

| Статус     | count после объединения | count в основной базе | Средний возраст после объединения, дн. | В основной базе, дн. |
|------------|-------------------------|-----------------------|----------------------------------------|----------------------|
| Pending    | 200 084                 | 200 084               | 366,431                                | 366,431              |
| InProgress | 199 766                 | 199 766               | 367,454                                | 367,454              |
| Completed  | 550 260                 | 550 260               | 367,059                                | 367,059              |
| Cancelled  | 49 890                  | 49 890                | 368,670                                | 368,670              |

Объединённый результат совпал с основной базой до единицы и до тысячной доли дня. COUNT на одном только Shard 0 дал 339 959 — треть правды.

Таблица 6 – Время агрегации: шарды параллельно и по очереди (медиана 5 прогонов)

| Шардов | Параллельно, мс | По очереди, мс |
|--------|-----------------|----------------|
| 3      | 77              | 160            |
| 4      | 65              | 168            |

**1. Можно ли выполнить такой запрос только на одном shard? Нет.**

В условии нет shard key, поэтому подходящие строки есть на каждом шарде, и любой один шард видит только свою часть данных. Запрос обязан уйти на все шарды — это scatter-gather.

**2. Где должны объединяться результаты?** В координаторе — в нашем случае в сервисе, в `DistributedTaskQueryService`. Базы шардов ничего не знают друг о друге и объединить результат не могут. Важно, что объединять можно не любые значения:

- COUNT и SUM складываются;
- MIN и MAX берутся по результатам шардов;
- AVG нельзя усреднять — шарды должны вернуть сумму и количество, а среднее считается уже после сложения. Тест `MergeStatuses_ShouldAddCountsAndComputeAverageFromTotals` показывает ошибку на простом примере: 10 задач по 10 дней на одном шарде и 90 задач по 1 дню на другом дают правильное среднее 1,9 дня, а «среднее средних» — 5.5. В реальных данных шарды похожи, и разница мала (366,431 против 366,410), но на неравномерных шардах она становится грубой ошибкой;

- COUNT(DISTINCT ...) и медиану так просто не объединить: нужно собирать сами значения или приближённые структуры вроде HyperLogLog.

**3. Что происходит со временем выполнения при увеличении количества shard'ов?** Растёт число обращений: каждый шард — это отдельное подключение, запрос и ответ. При последовательном опросе время складывается из времени всех шардов: 3 шарда — 160 мс, 4 шарда — 168 мс, хотя данных на каждом стало меньше. При параллельном опросе время определяет самый медленный шард: на 4 шардах оно даже уменьшилось (77 → 65 мс), потому что каждый шард считает меньше строк. Но у параллельного варианта свои цены: нагрузка на все серверы одновременно, число соединений растёт с числом шардов, а хвост задержек равен задержке

худшего шарда — чем шардов больше, тем вероятнее, что хотя бы один сейчас медленный.

## 4 JOIN между шардами

**Запрос сервиса.** Список задач для администратора — `TaskRepository.GetAllFilteredAsync`. EF Core строит JOIN четырёх таблиц:

### Листинг 5 – JOIN в запросе списка задач

```
SELECT t0.*, p.*, u.*, t1.*
FROM (SELECT t.* FROM tasks AS t WHERE ... ORDER BY t.created_at DESC LIMIT $1 OFFSET $2) AS t0
LEFT JOIN projects AS p ON t0.project_id = p.id
INNER JOIN users AS u ON t0.user_id = u.id
LEFT JOIN (SELECT t2.task_id, t2.tag_id, t3.* FROM task_tags AS t2
           INNER JOIN tags AS t3 ON t2.tag_id = t3.id) AS t1
ON t0.id = t1.task_id AND t0.created_at = t1.task_created_at
```

### Где находятся связанные данные.

Таблица 7 – Таблицы JOIN после шардирования

| Экземпляр     | Таблицы                          |
|---------------|----------------------------------|
| Shard 0, 1, 2 | tasks                            |
| Основная база | users, projects, tags, task_tags |

Тот же JOIN одним запросом на шарде невозможен:

### Листинг 6 – JOIN с проектами на шарде

```
ERROR: relation "projects" does not exist
LINE 1: SELECT t.title, p.name FROM tasks t LEFT JOIN projects p ON ...
```

Поэтому в сервисе JOIN выполняется на стороне приложения (`GET /api/shards/queries/users/{id}/tasks-with-projects`):

Таблица 8 – JOIN на стороне сервиса: задачи пользователя и названия проектов

| Шаг   | Где           | Запрос                                          | Строк передано | Время   |
|-------|---------------|-------------------------------------------------|----------------|---------|
| 1     | Shard 2       | задачи пользователя по user_id, 20 штук         | 20             | 1,9 мс  |
| 2     | основная база | projects WHERE id = ANY(...) по 14 собранным id | 14             | 15,4 мс |
| 3     | основная база | имя пользователя                                | 1              | 4,0 мс  |
| 4     | сервис        | соединение в памяти                             | —              | —       |
| Итого |               |                                                 | 35             | 24,2 мс |

Для сравнения: этот же JOIN в одной базе выполняется одним запросом за 2,0 мс.

**1. Что произойдёт, если связанные записи находятся на разных shard'ax?** Обычный JOIN их не найдёт: каждый PostgreSQL соединяет только свои таблицы, и строки с другого сервера для него не существуют. Результат будет неполным или запрос вообще упадёт, как на шарде выше. Связывать данные приходится вне базы, за несколько запросов.

**2. Можно ли выполнить такой JOIN обычным SQL-запросом к одному PostgreSQL?** Нет. Одним SQL-запросом можно соединять только таблицы одного сервера. Есть обходные механизмы — `postgres_fdw`, распределённые СУБД вроде Citus, — но это уже не обычный запрос: данные всё равно пересылаются между серверами, а планировщик плохо оценивает стоимость удалённых таблиц.

**3. Какие данные пришлось бы передавать между shard'ами?** Для application-side join: сначала строки одной стороны (20 задач), затем ключи соединения (14 `project_id`), затем найденные строки другой стороны (14 проектов). Для JOIN, у которого нет фильтра по shard key, — намного больше. Например, «все задачи проекта с именами исполнителей»: задачи проекта нужно собрать со всех шардов, так как все 1 000 проектов сервиса разбросаны по всем трём шардам. У самого большого проекта 277 задач на Shard 0, 293 на Shard 1 и 322 на Shard 2. Если бы и пользователи были шардированы по другому ключу, пришлось бы гонять между серверами целые промежуточные наборы строк.

**4. Можно ли выбрать shard key так, чтобы этот JOIN чаще выполнялся внутри одного shard?** Да, если связанные таблицы шардируются тем же ключом и лежат вместе (co-location):

- `task_tags` стоит хранить на шарде задачи: у связи тот же владелец, её можно положить по `user_id` задачи. Тогда соединение задач с тегами станет локальным;

- маленькие справочники — `tags`, `projects` — можно копировать на все шарды целиком (reference tables). Тогда JOIN с ними тоже выполняется на одном шарде, ценой синхронизации копий при изменениях;

– если бы главным запросом были «задачи проекта», подошёл бы `shard key project_id`. Но тогда распределёнными стали бы запросы «мои задачи», а у задачи без проекта не было бы ключа вообще.

`Shard key` выбирается под самые частые `JOIN` и фильтры: для `TaskManager` это запросы пользователя, поэтому `user_id` остаётся правильным выбором, а справочники нужно копировать на шарды.

## 5 ORDER BY и LIMIT

**Запрос сервиса.** Первая страница списка задач администратора — новые задачи сверху:

Листинг 7 – Запрос первой страницы новых задач

```
SELECT ... FROM tasks ORDER BY created_at DESC LIMIT 10 OFFSET 0
```

После шардирования его выполняет GET /api/shards/queries/newest: каждый шард возвращает свои 10 самых новых задач, сервис сливает 30 строк в общий порядок и отдаёт первые 10.

Таблица 9 – Первая страница после слияния: с какого шарда пришла каждая задача

| created_at          | Шард    | Задача       |
|---------------------|---------|--------------|
| 2026-09-13 22:50:56 | Shard 2 | Task #839460 |
| 2026-09-13 22:50:34 | Shard 2 | Task #91296  |
| 2026-09-13 22:50:03 | Shard 1 | Task #497509 |
| 2026-09-13 22:49:05 | Shard 2 | Task #606862 |
| 2026-09-13 22:48:24 | Shard 1 | Task #203638 |
| 2026-09-13 22:47:40 | Shard 2 | Task #743702 |
| 2026-09-13 22:47:00 | Shard 2 | Task #165000 |
| 2026-09-13 22:46:56 | Shard 0 | Task #487095 |
| 2026-09-13 22:46:51 | Shard 0 | Task #394139 |
| 2026-09-13 22:45:51 | Shard 1 | Task #6891   |

Слитая страница совпала с тем же запросом к основной базе — те же 10 задач в том же порядке.

Таблица 10 – Если взять первые 10 задач только с одного шарда

| Источник       | Задач из настоящей первой страницы |
|----------------|------------------------------------|
| только Shard 0 | 2 из 10                            |
| только Shard 1 | 3 из 10                            |
| только Shard 2 | 5 из 10                            |

**Почему недостаточно получить первые 100 записей только с одного shard.** Каждый шард знает порядок только своих строк. Самые новые задачи системы распределены по шардам случайно — shard key user\_id никак не связан со временем создания, — поэтому любой один шард содержит лишь часть глобальной первой страницы, а остальное вытеснили бы более новые

строки других шардов. Правильный ответ требует взять с каждого шарда его top-N и слить их.

При этом с каждого шарда нужно брать не «свою страницу», а весь префикс до конца запрошенной страницы. Шард не знает, сколько строк новее его строк лежит на других шардах: в худшем случае вся страница может оказаться с одного шарда. Поэтому для страницы с номером  $P$  каждый шард отдаёт  $P \times \text{size}$  строк:

Таблица 11 – Глубокие страницы: сколько строк передаётся ради 10 строк ответа

| Страница | Строк с каждого шарда | Передано в сервис | Отдано клиенту | Время, мс |
|----------|-----------------------|-------------------|----------------|-----------|
| 1        | 10                    | 30                | 10             | 23,5      |
| 10       | 100                   | 300               | 10             | 25,1      |
| 100      | 1 000                 | 3 000             | 10             | 29,8      |
| 1 000    | 10 000                | 30 000            | 10             | 87,8      |
| 5 000    | 50 000                | 150 000           | 10             | 178,7     |

Страница 100 после слияния тоже совпала с основной базой. Стоимость `OFFSET` в распределённой системе умножается на число шардов: ради 10 строк на странице 5 000 сервис получил 150 000 строк. Выход — `keyset-пагинация`: клиент передаёт `created_at` последней показанной задачи, и каждый шард отдаёт только 10 строк старше неё, на какой бы странице ни был пользователь.

## 6 Отказ одного шарда

На работающем сервисе остановлен Shard 2 (`docker compose stop postgres-shard-2`). В сервисе к этому моменту уже была обработка: ошибка подключения к шарду превращается в ответ 503 с кодом SHARD\_UNAVAILABLE, строка подключения содержит `Timeout=3`, чтобы не ждать 15 секунд по умолчанию. Опыт выявил ещё один случай — ошибку 57P01: `terminating connection due to administrator command` на соединении, которое пул Npgsql открыл до остановки сервера. Она тоже теперь считается отказом шарда, а пул соединений к нему сбрасывается.

Таблица 12 – Ответы сервиса до и во время отказа Shard 2

| Запрос                                            | До отказа    | Shard 2 остановлен    |
|---------------------------------------------------|--------------|-----------------------|
| GET /health                                       | 200, 0,011 с | 200, 0,009 с          |
| POST /api/auth/login (основная база)              | 200, 0,031 с | 200, 0,016 с          |
| GET /api/tasks (реплика)                          | 200, 0,293 с | 200, 0,015 с          |
| GET /api/projects (основная база)                 | 200, 0,106 с | 200, 0,010 с          |
| Задачи <code>gen_user_2</code> (Shard 0)          | 200, 0,030 с | 200, 0,031 с          |
| Задачи <code>gen_user_4</code> (Shard 2)          | 200, 0,034 с | 503, 1,292 с          |
| Создание задачи <code>gen_user_4</code> (Shard 2) | 201, 1,513 с | 503, 1,291 с          |
| Статистика по всем шардам                         | 200, 0,123 с | 503, 1,326 с          |
| Статистика, <code>allowPartial=true</code>        | 200, 0,084 с | 200, partial, 1,299 с |
| Новые задачи по всем шардам                       | 200, 0,046 с | 503, 1,318 с          |
| Новые задачи, <code>allowPartial=true</code>      | 200, 0,034 с | 200, partial, 1,305 с |

Листинг 8 – Ответ сервиса при обращении к недоступному шарду и неполная статистика

```
{ "code": "SHARD_UNAVAILABLE", "message": "Shard 2 is unavailable, the data stored there cannot be read or written now. Name or service not known" }

{ "partial": true, "total": 659715, "tasksPerShard": { "shard-0": 339959, "shard-1": 319756 },
  "shards": [ { "shard": 0, "available": true }, { "shard": 1, "available": true }, { "shard": 2, "available": false } ] }
```

После `docker compose start postgres-shard-2` сервис восстановился сам: задачи `gen_user_4` снова отдаются, статистика снова полная.

**1. Какие данные становятся недоступны?** Все задачи пользователей, которых router отправляет на Shard 2: 340 285 задач 3 390 пользователей —

треть данных. Остальные задачи, а также пользователи, проекты и теги в основной базе остаются доступны.

## **2. Какие endpoint'ы перестанут работать?**

- для пользователей Shard 2 — всё, что касается их задач: список, создание, изменение и удаление;
- для всех пользователей — распределённые запросы, которым нужны все шарды: общая статистика, общий список новых задач, поиск задачи по `id` без владельца.

Запросы пользователей Shard 0 и Shard 1 работают как раньше.

**3. Продолжит ли работать остальная часть сервиса?** Да. Вход в систему, проекты, теги, пользователи, `health check` и задачи двух других шардов ответили 200 с обычной скоростью. Шардирование изолировало отказ: пропала треть данных, а не весь сервис. При одной базе отказ сервера означал бы недоступность всего.

## **4. Что должен делать backend при обращении к недоступному shard?**

- Быстро вернуть понятную ошибку, а не зависать: здесь это 503 `SHARD_UNAVAILABLE` за 1,3 секунды с номером шарда. 503 честно сообщает, что проблема временная и на стороне сервера, и клиент может повторить запрос позже.
- Не притворяться, что данных нет: пустой список задач вместо ошибки пользователь принял бы за потерю данных.
- Для распределённых запросов — дать выбор. По умолчанию 503, а при `allowPartial=true` — результат доступных шардов с явной пометкой `partial: true` и списком недоступных шардов. Для дашборда это приемлемо, для финансового отчёта — нет.
- Не записывать данные «куда-нибудь ещё»: создание задачи на другом шарде нарушило бы `router`, и задача потом не нашлась бы.
- Сбрасывать пул соединений к упавшему серверу, писать предупреждение в лог и отражать состояние шардов в `health check`.

## 5. Как можно повысить отказоустойчивость?

- У каждого шарда — реплика со streaming replication (ЛР № 4) и автоматическое переключение на неё при отказе (Patroni, pg\_auto\_failover).
- Чтение с реплики шарда, пока основной экземпляр недоступен.
- Circuit breaker в сервисе: после серии ошибок временно не ходить на упавший шард и сразу отвечать 503.
- Мониторинг и оповещение по каждому шарду (health check /api/shards/queries/health плюс alert, как в ЛР № 3).
- Регулярные резервные копии каждого шарда и отработанная процедура восстановления.
- Кэш для самых частых чтений: Redis уже используется для списков задач.

## 7 Горячий шард

Сервис выполняет 20 000 настоящих запросов «задачи пользователя» (count + страница из 20 задач на шарде пользователя). Меняется только то, какие пользователи делают запросы. Данные на шардах всё время одни и те же: 34,0 / 32,0 / 34,0 % задач.

Таблица 13 – Распределение запросов по шардам при разном поведении пользователей

| Сценарий                                                 | Shard 0 | Shard 1        | Shard 2 |
|----------------------------------------------------------|---------|----------------|---------|
| Доля задач на шарде                                      | 34,00 % | 31,98 %        | 34,03 % |
| Все пользователи одинаково активны                       | 35,16 % | 30,56 %        | 34,27 % |
| Активность пропорциональна числу задач                   | 33,56 % | 32,70 %        | 33,74 % |
| 50 % запросов — популярный gen_user_1 (живёт на Shard 1) | 17,95 % | <b>65,02 %</b> | 17,03 % |
| 75 % запросов — gen_user_1                               | 8,95 %  | <b>82,34 %</b> | 8,70 %  |

Таблица 14 – Нагрузка на шарды при 75 % запросов популярного пользователя

| Шард    | Доля запросов  | Доля строк | Строк прочитано | Среднее время, мс | p95, мс     | Транзакций (pg_stat_database) |
|---------|----------------|------------|-----------------|-------------------|-------------|-------------------------------|
| Shard 0 | 8,95 %         | 34,00 %    | 35 800          | 1,14              | 1,72        | 5 365                         |
| Shard 1 | <b>82,34 %</b> | 31,98 %    | 329 380         | <b>1,89</b>       | <b>2,62</b> | <b>45 939</b>                 |
| Shard 2 | 8,70 %         | 34,03 %    | 34 820          | 1,13              | 1,74        | 5 232                         |

Картина «10 % / 15 % / 75 %» из методички воспроизведена почти буквально: при одинаковом объёме данных Shard 1 получил 82 % запросов и в 8,6 раза больше транзакций, чем соседи, а его запросы стали медленнее на 66 %. Каждый запрос этого пользователя считает 9 900 его задач, а не сотню, как у обычного пользователя.

### Возможные причины hot shard.

– **Неудачный shard key.** Если бы ключом был, например, status, большая часть задач (55 % — Completed) и почти все запросы попадали бы на один шард.

– **Неравномерное распределение данных.** Кольцо с малым числом виртуальных узлов дало одному из трёх шардов 39 % данных (ЛР № 5).

– **Популярный пользователь или tenant.** Главная причина в опыте: один ключ нельзя разделить, и вся его нагрузка ложится на один шард.

– **Определённый тип запросов.** Тяжёлые запросы — отчёты, экспорт, поиск без индекса — даже в небольшом количестве могут загрузить шард сильнее, чем тысячи лёгких запросов на соседних.

**Почему равное количество строк на shard'ах ещё не означает равномерную нагрузку.** Нагрузку создают запросы, а не строки. Количество строк описывает, как данные лежат, а нагрузка определяется тем, как часто и какими запросами к этим данным обращаются. Строки неравноценны: одни пользователи заходят раз в месяц, другие постоянно. Запросы тоже неравноценны: запрос тяжёлого пользователя читает в 100 раз больше строк. В опыте объём данных на шардах не изменился ни на строку, а доля запросов на Shard 1 выросла с 31 % до 82 %. Балансировать шарды нужно по фактической нагрузке — запросам, CPU, чтением с диска, — а для этого её нужно измерять на каждом шарде.

## 8 Итоговый анализ shard key

**Подходит ли `user_id` для TaskManager?** Да, с оговорками. Сервис — это задачи пользователей, и почти все его операции выполняются от имени конкретного пользователя.

### 8.1 Плюсы

Таблица 15 – Запросы, которые shard key делает эффективными

| Операция                                     | Почему эффективна                                                                       |
|----------------------------------------------|-----------------------------------------------------------------------------------------|
| Список своих задач с фильтрами и сортировкой | один шард, индекс ( <code>user_id</code> , <code>created_at</code> ) — 0,07 мс на шарде |
| Создание, изменение, удаление своей задачи   | один шард, транзакция в одной базе                                                      |
| Статистика своих задач                       | агрегация на одном шарде                                                                |
| Проверка владельца задачи                    | один шард, если известен <code>user_id</code>                                           |
| Изоляция отказов                             | отказ шарда затрагивает только его пользователей                                        |

Ключ не меняется, равномерно распределяет пользователей и позволяет добавлять шарды с переносом четверти данных (ЛР № 5).

### 8.2 Минусы

Таблица 16 – Запросы, которые становятся распределёнными

| Операция                                            | Что происходит                                             |
|-----------------------------------------------------|------------------------------------------------------------|
| Статистика и списки по всем задачам (администратор) | scatter-gather по всем шардам, объединение в сервисе       |
| Задача по <code>id</code> без владельца             | опрос всех шардов                                          |
| Задачи проекта                                      | все 1 000 проектов разбросаны по трём шардам               |
| Задачи с проектами, пользователями и тегами         | JOIN на стороне сервиса, несколько запросов к разным базам |
| Глубокая пагинация общего списка                    | каждый шард отдаёт весь префикс страницы                   |

### 8.3 Проблемные операции

Таблица 17 – Сложность операций после шардирования

| Операция | Сложность | Почему                                                                          |
|----------|-----------|---------------------------------------------------------------------------------|
| JOIN     | высокая   | таблицы на разных серверах; только application-side join или копии справочников |

| Операция         | Сложность | Почему                                                                                                                                |
|------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------|
|                  |           | на шардах                                                                                                                             |
| Aggregation      | средняя   | scatter-gather; COUNT и SUM объединяются просто, AVG — через сумму и количество, DISTINCT и медиана — трудно                          |
| ORDER BY + LIMIT | средняя   | слияние top-N; стоимость OFFSET умножается на число шардов                                                                            |
| Migration        | высокая   | миграцию схемы нужно применить на каждом шарде; нет общей транзакции, шарды могут оказаться в разных версиях                          |
| Backup           | средняя   | копия каждого шарда отдельно; согласованный снимок всех шардов на один момент не получается сам                                       |
| Восстановление   | высокая   | восстановленный шард отстаёт от остальных; ссылки между базами (задача → проект) могут указывать на данные из другого момента времени |
| Балансировка     | высокая   | горячий пользователь не делится между шардами; перенос данных нагружает шарды и требует окна, в котором router знает о переносе       |

## 8.4 Вывод

Если бы система проектировалась заново, shard key `user_id` стоило бы оставить. Он совпадает с тем, как сервисом пользуются: пользователь работает со своими задачами, и эти запросы — подавляющее большинство — выполняются на одном шарде так же быстро, как на отдельной базе. Альтернативы хуже. `project_id` делает распределёнными «мои задачи» и не подходит для задач без проекта. `id` задачи распределяет идеально, но превращает любой список в scatter-gather. `created_at` собирает всю свежую нагрузку на один шард.

Изменить стоило бы окружение ключа:

- таблицу `task_tags` хранить рядом с задачами на том же шарде;
- справочники `tags` и `projects` копировать на все шарды, чтобы основные JOIN стали локальными;
- административные отчёты по всем задачам перенести в отдельное аналитическое хранилище, куда данные шардов стекаются асинхронно, а не делать их scatter-gather к рабочим шардам;

— для очень крупных клиентов, если такие появятся, — выделенные шарды: горячий ключ при `user_id` неизбежен, и лучше заранее уметь выносить его отдельно.

## 9 Ответы на вопросы для самопроверки

1) **Чем single-shard query отличается от distributed query?** Single-shard query содержит shard key, поэтому router заранее знает единственный шард, где лежат нужные строки, и запрос выполняется как на обычной базе: одно подключение, один план, локальные индексы. Distributed query ключа не содержит или требует данных нескольких шардов: его отправляют на все нужные шарды, а ответы объединяет координатор. Это дороже по сети и времени и зависит от доступности всех шардов. В работе список задач пользователя — single-shard (0,07 мс на шарде), общая статистика — distributed (77 мс, три шарда).

2) **Почему JOIN между shard'ами сложнее обычного JOIN?** Обычный JOIN выполняет одна база: планировщик видит обе таблицы, их статистику и индексы и выбирает способ соединения. Таблицы на разных серверах одним SQL-запросом не соединить. JOIN приходится выполнять в приложении за несколько обращений: получить строки одной стороны, собрать ключи, запросить по ним строки другой стороны и соединить в памяти. Данные передаются по сети, согласованность двух запросов не гарантирует общая транзакция, а удачный план приходится придумывать вручную.

3) **Как выполнить COUNT(\*) по всем shard'ам?** Отправить COUNT(\*) на каждый шард — лучше параллельно — и сложить ответы в координаторе. Для группировок складываются значения с одинаковым ключом группы, для AVG шарды возвращают сумму и количество, а среднее считается после сложения. В работе:  $339\,959 + 319\,756 + 340\,285 = 1\,000\,000$ , что совпало с основной базой.

4) **Почему ORDER BY ... LIMIT требует объединения результатов?** Первые N строк глобального порядка лежат на разных шардах, и никакой шард не знает, какие из его строк входят в общий top-N. Нужно взять top-N с каждого шарда, слить их в общий порядок и снова отрезать N. Первые 10

задач одного шарда содержали лишь 2, 3 или 5 задач настоящей первой страницы. Для страницы Р каждый шард должен вернуть  $P \times \text{size}$  строк, поэтому глубокие страницы дороги: 150 000 переданных строк ради 10 на странице 5 000.

**5) Что произойдёт при отказе одного shard?** Станут недоступны данные этого шарда — треть задач в опыте — и все запросы, которым нужны все шарды. Остальная часть сервиса продолжит работать: вход, проекты и задачи пользователей других шардов отвечали 200. Корректный сервис быстро возвращает понятную ошибку (503 SHARD\_UNAVAILABLE), по явному запросу отдаёт неполный результат с пометкой и восстанавливается сам, когда шард возвращается.

**6) Что такое hot shard?** Шард, который получает непропорционально большую долю нагрузки и становится узким местом системы. В опыте Shard 1 при 32 % данных принял 82 % запросов и 45 939 транзакций против 5 365 у соседей, а его запросы стали на 66 % медленнее. Причины: популярный пользователь или tenant, неудачный shard key, неравномерное распределение данных или тяжёлые запросы к одному шарду.

**7) Почему выбор shard key влияет на JOIN?** JOIN выполняется локально, только если все соединяемые строки лежат на одном шарде. Это возможно, когда связанные таблицы шардированы тем же ключом (co-location) или скопированы на все шарды. При user\_id задачи пользователя и связи этих задач с тегами можно положить вместе, и их JOIN станет локальным. А задачи одного проекта разбросаны по всем шардам — все 1 000 проектов в опыте, — поэтому любой JOIN «через проект» остаётся распределённым.

**8) Какие операции особенно плохо подходят для распределённых данных?**

- JOIN между таблицами, шардированными разными ключами;
- транзакции, меняющие данные на нескольких шардах;

- глобальные ограничения уникальности и внешние ключи между базами;
- глубокая пагинация через OFFSET;
- COUNT (DISTINCT), медианы и перцентили по всем данным;
- согласованные резервные копии всех шардов и изменения схемы на всех шардах сразу.

9) **Что произойдёт при изменении количества shard'ов?** Часть ключей начнёт соответствовать другим шардам, и данные этих ключей нужно перенести до переключения router'a. При  $\text{hash \% N}$  переносится большая часть данных — 75 % при переходе с трёх шардов на четыре. При consistent hashing — примерно доля нового шарда: 24–27 %, и только на новый шард (ЛР № 5). Пока идёт перенос, сервис должен знать, где сейчас данные переносимых пользователей, иначе чтение уйдёт на шард, где строк уже нет.

10) **Чем шардирование отличается от репликации?** Репликация копирует одни и те же данные целиком на несколько серверов. Она повышает отказоустойчивость и позволяет распределить чтение, но каждая копия хранит всё, а запись по-прежнему идёт на один Primary. Шардирование делит данные на части: каждый шард хранит только своё подмножество. Это масштабирует объём данных и запись, но каждая часть существует в одном экземпляре, и отказ шарда означает потерю доступа к его данным. На практике их совмещают: у каждого шарда есть свои реплики.

11) **Может ли шардирование сделать отдельный SQL-запрос быстрее?** Иногда да, но это не главная цель. Запрос по shard key работает с меньшей таблицей и меньшими индексами, которые лучше помещаются в память; агрегация параллельно на четырёх шардах в опыте оказалась быстрее, чем на трёх (65 против 77 мс). Однако запросы без shard key становятся медленнее и сложнее. Задача с проектами одним JOIN в одной базе — 2 мс, через шард и основную базу — 24 мс. Глубокая страница общего списка требует передачи сотен тысяч строк. Шардирование

масштабирует объём данных и число одновременных операций, а скорость отдельного запроса обеспечивают индексы и хороший SQL.

## ЗАКЛЮЧЕНИЕ

В работе проанализированы реальные запросы сервиса TaskManager после шардирования задач по `user_id` на три PostgreSQL. Для каждого вида запроса поведение проверено на работающем сервисе: для этого добавлен сервис распределённых запросов с endpoint'ами `/api/shards/queries/...`

Запрос задач пользователя остался single-shard: он выполняется индексом одного шарда за 0,07 мс, на других шардах этих строк нет. Агрегация по всем задачам стала scatter-gather: шарды возвращают свои части, сервис складывает их, и результат совпал с основной базой до единицы. AVG объединяется только через сумму и количество. Время при последовательном опросе растёт с числом шардов, а при параллельном определяется самым медленным шардом. JOIN задач с проектами одним SQL-запросом на шарде невозможен и выполняется в сервисе тремя запросами к двум базам: 24 мс вместо 2 мс в одной базе. ORDER BY + LIMIT требует слияния top-N со всех шардов: один шард содержал лишь 2–5 задач из настоящей первой десятки, а глубокие страницы стоят сотни тысяч переданных строк.

Отказ Shard 2 сделал недоступной треть задач и распределённые запросы, но не сервис целиком. Сервис отвечал 503 SHARD\_UNAVAILABLE за 1,3 секунды, по запросу отдавал неполную статистику с пометкой и сам восстановился после возврата шарда. Опыт выявил и исправил ошибку обработки оборванных соединений из пула. Популярный пользователь, делающий 75 % запросов, превратил один шард в горячий — 82 % запросов при 32 % данных — и подтвердил, что равный объём данных не означает равную нагрузку.

Shard key `user_id` признан подходящим для сервиса: самые частые операции остаются локальными. Распределёнными становятся административные отчёты, запросы по проекту и JOIN со справочниками. Их

стоит решать копированием справочников на шарды, хранением связей задач рядом с задачами и отдельным аналитическим хранилищем.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 PostgreSQL 16 Documentation. 28.2. The Cumulative Statistics System [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/monitoring-stats.html> (дата обращения: 15.09.2026).

2 PostgreSQL 16 Documentation. Appendix A. PostgreSQL Error Codes [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/errcodes-appendix.html> (дата обращения: 15.09.2026).

3 PostgreSQL 16 Documentation. F.36. postgres\_fdw [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/postgres-fdw.html> (дата обращения: 15.09.2026).

4 Npgsql Documentation. Connection String Parameters [Электронный ресурс]. – URL: <https://www.npgsql.org/doc/connection-string-parameters.html> (дата обращения: 15.09.2026).

5 Citus Documentation [Электронный ресурс]. – URL: <https://docs.citusdata.com/> (дата обращения: 15.09.2026).

6 Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. – СПб. : Питер, 2018. – 640 с.

7 Лабораторная работа № 6. Что происходит с backend-сервисом после шардирования : методические указания. – Сириус : Колледж Университета «Сириус», 2026. – 5 с.

8 VHS — инструмент записи терминала [Электронный ресурс]. – URL: <https://github.com/charmbracelet/vhs> (дата обращения: 15.09.2026).

## ПРИЛОЖЕНИЕ А

### Сервис распределённых запросов

Класс `DistributedTaskQueryService`: scatter-gather по шардам с замером времени и обработкой недоступных шардов, JOIN на стороне сервиса и генератор нагрузки для опыта с горячим шардом (разделы 3–7).

#### Листинг А.1 – `DistributedTaskQueryService.cs`

```
using System.Diagnostics;
using Microsoft.Extensions.Logging;
using TaskManager.Application.Exceptions;
using TaskManager.Application.Interfaces;
using TaskManager.Application.Sharding;

namespace TaskManager.Application.Services;

/// <summary>
/// Scatter-gather over task shards: the query is sent to the needed shards, the service merges
the answers.
/// </summary>
public class DistributedTaskQueryService : IDistributedTaskQueryService
{
    private readonly IShardedTaskStore _store;
    private readonly ILogger<DistributedTaskQueryService> _logger;

    /// <summary>
    /// Initializes a new instance of the <see cref="DistributedTaskQueryService"/> class.
    /// </summary>
    public DistributedTaskQueryService(IShardedTaskStore store,
    ILogger<DistributedTaskQueryService> logger)
    {
        _store = store;
        _logger = logger;
    }

    /// <inheritdoc/>
    public async Task<DistributedStats> GetStatsAsync(bool parallel, bool allowPartial,
    CancellationToken cancellationToken = default)
    {
        var total = Stopwatch.StartNew();
        var now = DateTime.UtcNow;
        var shards = await ActiveShardsAsync(cancellationToken);
        var answers = await ScatterAsync(
            shards, parallel, allowPartial,
            (shard, ct) => _store.AggregateByStatusAsync(shard, now, ct),
            rows => rows.Count,
            cancellationToken);

        var aggregates = answers.Results.Select(r => r.Value).ToList();
        return new DistributedStats
        {
            Parallel = parallel,
            Shards = answers.Calls,
            Partial = answers.Calls.Any(c => !c.Available),
            Statuses = DistributedMerge.MergeStatuses(aggregates),
            TasksPerShard = answers.Results.ToDictionary(r => $"shard-{r.Shard}", r =>
r.Value.Sum(a => a.Count)),
            Total = aggregates.Sum(a => a.Sum(x => x.Count)),
            TotalMilliseconds = Elapsed(total)
        };
    }

    /// <inheritdoc/>
    public async Task<DistributedPage> GetNewestAsync(int page, int pageSize, bool
allowPartial, CancellationToken cancellationToken = default)
```

```

    {
        var total = Stopwatch.StartNew();
        var take = page * pageSize;
        var shards = await ActiveShardsAsync(cancellationToken);

        // A shard does not know how many newer rows the other shards have, so each must return
        the whole
        // prefix up to the end of the page, not just its own "page N"
        var answers = await ScatterAsync(
            shards, parallel: true, allowPartial,
            (shard, ct) => _store.GetNewestAsync(shard, take, ct),
            rows => rows.Count,
            cancellationToken);

        return new DistributedPage
        {
            Page = page,
            PageSize = pageSize,
            RowsRequestedPerShard = take,
            Shards = answers.Calls,
            Partial = answers.Calls.Any(c => !c.Available),
            Items = DistributedMerge.MergeNewest(answers.Results.Select(r => (r.Shard,
r.Value)), (page - 1) * pageSize, pageSize),
            TotalMilliseconds = Elapsed(total)
        };
    }

    /// <inheritdoc>
    public async Task<DistributedPage> GetNewestFromOneShardAsync(int shard, int pageSize,
CancellationToken cancellationToken = default)
    {
        var total = Stopwatch.StartNew();
        var rows = await _store.GetNewestAsync(shard, pageSize, cancellationToken);
        return new DistributedPage
        {
            Page = 1,
            PageSize = pageSize,
            RowsRequestedPerShard = pageSize,
            Shards = new List<ShardCall> { new(shard, true, rows.Count, Elapsed(total)) },
            Items = DistributedMerge.MergeNewest(new[] { (shard, rows) }, 0, pageSize),
            TotalMilliseconds = Elapsed(total)
        };
    }

    /// <inheritdoc>
    public async Task<DistributedJoin> GetUserTasksWithProjectsAsync(Guid userId, int pageSize,
CancellationToken cancellationToken = default)
    {
        var total = Stopwatch.StartNew();
        var topology = await _store.GetTopologyAsync(cancellationToken);
        var shard = topology.CreateRouter().GetShard(userId);
        var result = new DistributedJoin { Shard = shard };

        // 1. Tasks: one shard, found by the shard key
        var step = Stopwatch.StartNew();
        var tasks = await _store.GetByKeyAsync(shard, userId, 1, pageSize, cancellationToken);
        result.Steps.Add(new JoinStep($"shard-{shard}", "SELECT ... FROM tasks WHERE user_id =
@user ORDER BY created_at DESC LIMIT @n", tasks.Items.Count, Elapsed(step)));
        result.Shards.Add(new ShardCall(shard, true, tasks.Items.Count, Elapsed(step)));

        // 2. Projects: another database, so the JOIN turns into a second query with the
        collected ids
        step.Restart();
        var projectIds = tasks.Items.Where(t => t.ProjectId.HasValue).Select(t =>
t.ProjectId!.Value).Distinct().ToList();
        var projects = await _store.GetProjectNamesAsync(projectIds, cancellationToken);
        result.Steps.Add(new JoinStep("main", $"SELECT id, name FROM projects WHERE id =
ANY(@ids), {projectIds.Count} ids", projects.Count, Elapsed(step)));

        // 3. The user: also in the main database
        step.Restart();
        result.Username = await _store.GetUsernameAsync(userId, cancellationToken);
        result.Steps.Add(new JoinStep("main", "SELECT username FROM users WHERE id = @user",
result.Username is null ? 0 : 1, Elapsed(step)));
    }

```

```

// 4. The JOIN itself happens here, in the memory of the service
result.Items = tasks.Items
    .Select(t => new TaskWithProject(
        t.Id, t.Title, t.Status.ToString(), t.CreatedAt, t.ProjectId,
        t.ProjectId is { } id && projects.TryGetValue(id, out var name) ? name : null))
    .ToList();
result.TotalMilliseconds = Elapsed(total);
return result;
}

/// <inheritdoc>
public async Task<WorkloadResult> RunWorkloadAsync(
    WorkloadDistribution distribution, int requests, Guid? hotUserId, double
hotSharePercent,
    CancellationToken cancellationToken = default)
{
    var topology = await _store.GetTopologyAsync(cancellationToken);
    var router = topology.CreateRouter();
    var shardCount = topology.ActiveShards;

    // Users and their task counts, as they are really stored on the shards
    var tasksByUser = new Dictionary<Guid, long>();
    var keys = new long[shardCount];
    var records = new long[shardCount];
    for (var shard = 0; shard < shardCount; shard++)
    {
        var counts = await _store.CountByKeyAsync(shard, cancellationToken);
        keys[shard] = counts.Count;
        records[shard] = counts.Values.Sum();
        foreach (var (user, count) in counts)
        {
            tasksByUser[user] = count;
        }
    }

    var users = tasksByUser.Keys.OrderBy(u => u).ToArray();
    var cumulative = new double[users.Length];
    var sum = 0d;
    for (var i = 0; i < users.Length; i++)
    {
        sum += tasksByUser[users[i]];
        cumulative[i] = sum;
    }

    var hot = hotUserId ?? tasksByUser.MaxBy(t => t.Value).Key;
    var random = new Random(42);
    Guid Pick()
    {
        switch (distribution)
        {
            case WorkloadDistribution.ByTasks:
                var index = Array.BinarySearch(cumulative, random.NextDouble() * sum);
                return users[Math.Min(index < 0 ? ~index : index, users.Length - 1)];
            case WorkloadDistribution.HotUser when random.NextDouble() * 100 <
hotSharePercent:
                return hot;
            default:
                return users[random.Next(users.Length)];
        }
    }

    var picks = Enumerable.Range(0, requests).Select(_ => Pick()).ToArray();
    var times = Enumerable.Range(0, shardCount).Select(_ => new List<double>()).ToArray();
    var rows = new long[shardCount];
    var sync = new object();
    using var gate = new SemaphoreSlim(16);
    var total = Stopwatch.StartNew();

    await Task.WhenAll(picks.Select(async user =>
    {
        await gate.WaitAsync(cancellationToken);
        try
        {

```

```

        var shard = router.GetShard(user);
        var watch = Stopwatch.StartNew();
        var page = await _store.GetByKeyAsync(shard, user, 1, 20, cancellationToken);
        lock (sync)
        {
            times[shard].Add(watch.Elapsed.TotalMilliseconds);
            rows[shard] += page.Items.Count;
        }
    }
    finally
    {
        gate.Release();
    }
}));

return new WorkloadResult
{
    Distribution = distribution,
    HotUserId = distribution == WorkloadDistribution.HotUser ? hot : null,
    HotSharePercent = distribution == WorkloadDistribution.HotUser ? hotSharePercent :
null,
    HotUserShard = distribution == WorkloadDistribution.HotUser ? router.GetShard(hot)
: null,
    Requests = requests,
    Seconds = Math.Round(total.Elapsed.TotalSeconds, 2),
    Shards = Enumerable.Range(0, shardCount).Select(shard =>
    {
        var sorted = times[shard].OrderBy(t => t).ToList();
        return new WorkloadShard(
            shard,
            sorted.Count,
            Math.Round(100.0 * sorted.Count / requests, 2),
            Math.Round(100.0 * keys[shard] / keys.Sum(), 2),
            Math.Round(100.0 * records[shard] / records.Sum(), 2),
            rows[shard],
            sorted.Count == 0 ? 0 : Math.Round(sorted.Average(), 2),
            sorted.Count == 0 ? 0 : Math.Round(sorted[(int)Math.Min(sorted.Count - 1,
Math.Ceiling(sorted.Count * 0.95) - 1)], 2));
        }).ToList()
    );
}

/// <inheritdoc>
public async Task<List<ShardCall>> GetHealthAsync(Cancellation token cancellationToken =
default)
{
    var shards = await ActiveShardsAsync(cancellationToken);
    var answers = await ScatterAsync(
        shards, parallel: true, allowPartial: true,
        async (shard, ct) => { await _store.PingAsync(shard, ct); return 0; },
        _ => 0,
        cancellationToken);
    return answers.Calls;
}

private async Task<IReadOnlyList<int>> ActiveShardsAsync(Cancellation token
cancellationToken)
{
    var topology = await _store.GetTopologyAsync(cancellationToken);
    return Enumerable.Range(0, topology.ActiveShards).ToList();
}

private async Task<(List<ShardCall> Calls, List<(int Shard, T Value)> Results)>
ScatterAsync<T>(
    IReadOnlyList<int> shards,
    bool parallel,
    bool allowPartial,
    Func<int, Cancellation token, Task<T>> query,
    Func<T, long> rows,
    Cancellation token cancellationToken)
{
    async Task<(ShardCall Call, T? Value)> Run(int shard)
    {
        var watch = Stopwatch.StartNew();
        try
        {
            var value = await query(shard, cancellationToken);

```

```

        return (new ShardCall(shard, true, rows(value), Elapsed(watch)), value);
    }
    catch (ShardUnavailableException ex)
    {
        return (new ShardCall(shard, false, 0, Elapsed(watch), ex.Message), default);
    }
}

var answers = new List<ShardCall Call, T? Value>();
if (parallel)
{
    answers.AddRange(await Task.WhenAll(shards.Select(Run)));
}
else
{
    foreach (var shard in shards)
    {
        answers.Add(await Run(shard));
    }
}

var failed = answers.Where(a => !a.Call.Available).Select(a => a.Call.Shard).ToList();
if (failed.Count > 0)
{
    _logger.LogWarning("Distributed query: shards {Shards} did not answer, partial
result allowed: {Allowed}", failed, allowPartial);
    if (!allowPartial)
    {
        throw new ShardUnavailableException(failed);
    }
}

return (
    answers.Select(a => a.Call).OrderBy(c => c.Shard).ToList(),
    answers.Where(a => a.Call.Available).Select(a => (a.Call.Shard,
a.Value!)).OrderBy(r => r.Shard).ToList());
}

private static double Elapsed(Stopwatch watch) =>
Math.Round(watch.Elapsed.TotalMilliseconds, 2);
}

```

## ПРИЛОЖЕНИЕ Б

### Объединение результатов шардов

Модели распределённых запросов и класс DistributedMerge: слияние агрегатов через суммы и количества и слияние top-N для ORDER BY + LIMIT (разделы 3 и 5).

#### Листинг Б.1 – DistributedQueries.cs

```
using TaskManager.Domain.Entities;
using TaskManager.Domain.Enums;

namespace TaskManager.Application.Sharding;

/// <summary>
/// Partial aggregate of one shard: enough to merge exactly (sums and counts, not averages).
/// </summary>
/// <param name="Status">The task status.</param>
/// <param name="Count">Tasks with the status.</param>
/// <param name="Overdue">Overdue tasks among them.</param>
/// <param name="AgeDaysSum">Sum of task ages in days, to compute the average after the
merge.</param>
public sealed record StatusAggregate(int Status, long Count, long Overdue, double AgeDaysSum);

/// <summary>
/// How one shard answered a distributed query.
/// </summary>
/// <param name="Shard">The shard number.</param>
/// <param name="Available">Whether the shard answered.</param>
/// <param name="Rows">Rows the shard sent to the service.</param>
/// <param name="Milliseconds">How long the shard took, including the connection.</param>
/// <param name="Error">The reason when the shard did not answer.</param>
public sealed record ShardCall(int Shard, bool Available, long Rows, double Milliseconds,
string? Error = null);

/// <summary>
/// Merged value of one status.
/// </summary>
/// <param name="Status">The task status.</param>
/// <param name="Count">Tasks with the status on all answered shards.</param>
/// <param name="Overdue">Overdue tasks.</param>
/// <param name="AverageAgeDays">Correct average age: total sum divided by total count.</param>
/// <param name="AverageOfShardAveragesDays">Wrong average: plain mean of shard averages, shown
for comparison.</param>
public sealed record StatusStat(string Status, long Count, long Overdue, double AverageAgeDays,
double AverageOfShardAveragesDays);

/// <summary>
/// Common part of every distributed query result.
/// </summary>
public abstract class DistributedResult
{
    /// <summary>Gets or sets how each shard answered.</summary>
    public List<ShardCall> Shards { get; set; } = new();

    /// <summary>Gets or sets a value indicating whether some shards did not answer and the
result is incomplete.</summary>
    public bool Partial { get; set; }

    /// <summary>Gets or sets the total time of the request in the service, in
milliseconds.</summary>
    public double TotalMilliseconds { get; set; }
}

/// <summary>
```

```

/// COUNT and AVG by status over all shards.
/// </summary>
public class DistributedStats : DistributedResult
{
    /// <summary>Gets or sets a value indicating whether the shards were queried in
    parallel.</summary>
    public bool Parallel { get; set; }

    /// <summary>Gets or sets the merged values by status.</summary>
    public List<StatusStat> Statuses { get; set; } = new();

    /// <summary>Gets or sets the number of tasks per shard, before the merge.</summary>
    public Dictionary<string, long> TasksPerShard { get; set; } = new();

    /// <summary>Gets or sets all tasks on the answered shards.</summary>
    public long Total { get; set; }
}

/// <summary>
/// A page of the newest tasks over all shards: ORDER BY created_at DESC LIMIT.
/// </summary>
public class DistributedPage : DistributedResult
{
    /// <summary>Gets or sets the page number.</summary>
    public int Page { get; set; }

    /// <summary>Gets or sets the page size.</summary>
    public int PageSize { get; set; }

    /// <summary>Gets or sets how many rows each shard had to return: OFFSET + LIMIT.</summary>
    public int RowsRequestedPerShard { get; set; }

    /// <summary>Gets or sets the page after the merge.</summary>
    public List<ShardedTaskRow> Items { get; set; } = new();
}

/// <summary>
/// The user's tasks from their shard joined with project names from the main database in the
/// service.
/// </summary>
public class DistributedJoin : DistributedResult
{
    /// <summary>Gets or sets the shard of the user.</summary>
    public int Shard { get; set; }

    /// <summary>Gets or sets the user name read from the main database.</summary>
    public string? Username { get; set; }

    /// <summary>Gets or sets the steps of the join and their cost.</summary>
    public List<JoinStep> Steps { get; set; } = new();

    /// <summary>Gets or sets the joined rows.</summary>
    public List<TaskWithProject> Items { get; set; } = new();
}

/// <summary>
/// One round trip of an application-side join.
/// </summary>
/// <param name="Database">Where the query went.</param>
/// <param name="Query">What was asked.</param>
/// <param name="Rows">Rows transferred to the service.</param>
/// <param name="Milliseconds">How long it took.</param>
public sealed record JoinStep(string Database, string Query, long Rows, double Milliseconds);

/// <summary>
/// A task row with the shard it came from.
/// </summary>
public sealed record ShardedTaskRow(int Shard, Guid Id, Guid UserId, string Title, string
Status, DateTime CreatedAt);

```

```

/// <summary>
/// A task joined with its project.
/// </summary>
public sealed record TaskWithProject(Guid Id, string Title, string Status, DateTime CreatedAt,
Guid? ProjectId, string? ProjectName);

/// <summary>
/// Pure merge logic of distributed queries.
/// </summary>
public static class DistributedMerge
{
    /// <summary>
    /// Merges per-shard aggregates: counts and sums are added, the average is computed from
    the totals.
    /// </summary>
    public static List<StatusStat>
MergeStatuses(IReadOnlyCollection<IReadOnlyList<StatusAggregate>> shards)
    {
        return shards
            .SelectMany(s => s)
            .GroupBy(a => a.Status)
            .OrderBy(g => g.Key)
            .Select(g =>
            {
                var count = g.Sum(a => a.Count);
                return new StatusStat(
                    ((TaskItemStatus)g.Key).ToString(),
                    count,
                    g.Sum(a => a.Overdue),
                    count == 0 ? 0 : Math.Round(g.Sum(a => a.AgeDaysSum) / count, 3),
                    Math.Round(g.Where(a => a.Count > 0).Average(a => a.AgeDaysSum / a.Count),
3));
            })
            .ToList();
    }

    /// <summary>
    /// Merges the newest rows of every shard into one global order and cuts the requested
    page.
    /// Every shard must have returned at least <paramref name="skip"/> + <paramref
    name="take"/> rows.
    /// </summary>
    public static List<ShardedTaskRow> MergeNewest(
        IEnumerable<(int Shard, IReadOnlyList<TaskItem> Rows)> shards, int skip, int take)
    {
        return shards
            .SelectMany(s => s.Rows.Select(t => new ShardedTaskRow(s.Shard, t.Id, t.UserId,
t.Title, t.Status.ToString(), t.CreatedAt)))
            .OrderByDescending(r => r.CreatedAt)
            .ThenByDescending(r => r.Id)
            .Skip(skip)
            .Take(take)
            .ToList();
    }
}

```

## ПРИЛОЖЕНИЕ В

### Запросы к отдельному шарду

Часть класса `ShardedTaskStore`: запросы, из которых складываются распределённые запросы (разделы 3 и 5).

#### Листинг В.1 – `ShardedTaskStore.Queries.cs`

```
using Dapper;
using Npgsql;
using TaskManager.Application.Sharding;
using TaskManager.Domain.Entities;

namespace TaskManager.Infrastructure.Repositories;

/// <summary>
/// Queries that are parts of distributed requests: each runs on one shard, the service merges
/// the results.
/// </summary>
public partial class ShardedTaskStore
{
    /// <inheritdoc/>
    public async Task<IReadOnlyList<StatusAggregate>> AggregateByStatusAsync(
        int shard, DateTime now, CancellationToken cancellationToken = default)
    {
        return await OnShardAsync(shard, async connection =>
        {
            var rows = await connection.QueryAsync<StatusAggregate>(new CommandDefinition(@"
                SELECT status AS Status,
                count(*) AS Count,
                count(*) FILTER (WHERE due_date < @Now AND status IN (0, 1)) AS Overdue,
                coalesce(sum(extract(epoch FROM (@Now - created_at)) / 86400),
0)::float8 AS AgeDaysSum
            FROM tasks
            GROUP BY status
            ORDER BY status",
                new { Now = now }, cancellationToken: cancellationToken));
            return rows.ToList();
        }, cancellationToken);
    }

    /// <inheritdoc/>
    public async Task<IReadOnlyList<TaskItem>> GetNewestAsync(int shard, int take,
        CancellationToken cancellationToken = default)
    {
        return await OnShardAsync(shard, async connection =>
        {
            var rows = await connection.QueryAsync<TaskRow>(new CommandDefinition(
                $"SELECT {Columns} FROM tasks ORDER BY created_at DESC, id DESC LIMIT @Take",
                new { Take = take }, cancellationToken: cancellationToken));
            return rows.Select(r => r.ToEntity()).ToList();
        }, cancellationToken);
    }

    /// <inheritdoc/>
    public async Task<IReadOnlyDictionary<Guid, string>> GetProjectNamesAsync(
        IReadOnlyCollection<Guid> projectIds, CancellationToken cancellationToken = default)
    {
        if (projectIds.Count == 0)
        {
            return new Dictionary<Guid, string>();
        }

        await using var connection = new NpgsqlConnection(_mainConnectionString);
        var rows = await connection.QueryAsync<(Guid Id, string Name)>(new CommandDefinition(
            "SELECT id, name FROM projects WHERE id = ANY(@Ids)", new { Ids =
projectIds.ToArray() },
            cancellationToken: cancellationToken));
        return rows.ToDictionary(r => r.Id, r => r.Name);
    }
}
```

```

    }

    /// <inheritdoc/>
    public async Task<string?> GetUsernameAsync(Guid userId, CancellationToken
cancellationToken = default)
    {
        await using var connection = new NpgsqlConnection(_mainConnectionString);
        return await connection.ExecuteScalarAsync<string?>(new CommandDefinition(
            "SELECT username FROM users WHERE id = @Id", new { Id = userId },
            cancellationToken: cancellationToken));
    }

    /// <inheritdoc/>
    public async Task PingAsync(int shard, CancellationToken cancellationToken = default)
    {
        await OnShardAsync(shard, async connection =>
        {
            await connection.ExecuteScalarAsync<int>(new CommandDefinition("SELECT 1",
            cancellationToken: cancellationToken));
        }, cancellationToken);
    }
}

```